機器學習實作｜期中考小抄 考前重點與 API 補強版（一）觀念與模型

範圍：Lectures 1–6 投影片｜單選 25–50 題｜考觀念不考推導＋Python 程式解讀

1 學習類型與任務

類型	資料	代表任務
監督式	有標籤 y	分類（離散）、迴歸（連續）
非監督式	無標籤	分群、降維
半監督	少量標籤＋大量無標籤	省標註成本
強化式	獎勵訊號	代理人試錯、最大化總獎勵

- 樣本＝一列、特徵＝一欄、標籤 / ground truth＝答案
- **可訓練參數**（$\boldsymbol{w}, b$，從資料學）vs **超參數**（η、epoch、C、k，人先設）
- **特徵選取**挑子集（欄位意義不變）vs **特徵提取**算出新特徵（如 PCA）
- Epoch＝全部訓練資料跑完一輪；**反向傳播**＝用連鎖律往回算 loss 對參數的梯度；更新由最佳化器負責
- **No Free Lunch**：沒有在所有問題都最好的模型 → 要比較多個演算法
- 流程：先切分 → 前處理只 fit 訓練集 → 訓練 / 選模型（CV）→ 測試集評估一次

2 三個線性分類器的比較

	感知器 PLA	Adaline	邏輯迴歸
淨輸入	$z = \boldsymbol{w}^T\boldsymbol{x} + b$（三者相同）		
激活	階梯函數（不可微）	恆等 $\sigma(z) = z$	sigmoid $\frac{1}{1+e^{-z}}$
損失	依誤分類更新	**MSE**（凸）	**二元交叉熵 BCE**（log loss、負對數概似）
更新時機	只有分錯才更新	依連續誤差更新（分對也可能更新）	同 Adaline
輸出	0 / 1	線性值；0/1 標籤門檻 0.5	機率 $p(y{=}1\|\boldsymbol{x})$

- 決策邊界為超平面；PLA / Logistic：$z = 0$；本課 Adaline 門檻 .5：$z = .5$。原始閾值與 bias：$b = -\theta$
- PLA（0/1 標籤）：$\Delta w = \eta(y - \hat{y})x$、$\Delta b = \eta(y - \hat{y})$；分對不更新。**Novikoff**：線性可分保證有限步收斂；不可分需設 epoch 上限
- Adaline 從「對或錯」改成「差多少」→ 可微分 → 能用梯度下降
- 邏輯迴歸更新式和 Adaline 形式相同：$\boldsymbol{w} \leftarrow \boldsymbol{w} + \eta\frac{1}{n}\boldsymbol{X}^T(\boldsymbol{y} - \boldsymbol{a})$，差在 a 是 sigmoid 輸出
- 多元分類：**OvR / OvA**（K 個二元分類器；互斥多類取最高分，多標籤各自切門檻，機率不必加總為 1）vs **softmax**（類別互斥，機率和＝1）

3 邏輯迴歸細節

- 名字有 regression，做的是**分類**；線性建模的是 logit，不是 p
- 勝算 $\frac{p}{1-p}$（$p{=}0.8 \to 4$）；logit $= \ln\frac{p}{1-p}$ 把 $(0,1)$ 映到全實數。**為什麼要繞這圈**：線性式 $\boldsymbol{w}^T\boldsymbol{x} + b$ 的值域是整條實數線、機率卻只在 $(0,1)$，所以改成「用線性式建模 logit」，再用其反函數 sigmoid 轉回機率，$\sigma(0) = 0.5$
- sigmoid 輸出是**條件機率** $p(y{=}1|\boldsymbol{x})$＝「已知特徵時屬於正類」的機率，不是預測正確的機率
- BCE：$L = -\frac{1}{n}\sum[y\ln a + (1-y)\ln(1-a)]$；單筆 $y{=}1$ 時 $-\ln a$，**自信地答錯**（$a \to 0$）損失 $\to \infty$
- 損失辨識：Adaline 的 MSE 比較 y 與線性輸出 z；邏輯迴歸的 BCE 比較 y 與機率 $a = \sigma(z)$，**不要把** z **當機率**。Adaline：$y = 1, z = 5$ 已分對，平方誤差仍為 16，會把 z 拉回 1
- 決策邊界 $z = 0$ 仍是**線性**（sigmoid 不會讓邊界變彎）；要非線性得做特徵工程（如 $|x_1|$ 表達離 0 多遠、x^2）
- sklearn：`predict_proba` 給機率、`C` 越小正則化越強

補強 A｜前處理 API（L5）

函式 / 類別	功能與陷阱
LabelEncoder	用於 y；inverse_transform 還原標籤
OneHotEncoder	輸入二維；drop='first' 去冗餘欄
FunctionTransformer	把函式包為轉換器；np.log1p(X)=ln(1+X)
ColumnTransformer	分欄轉換：(name, transformer, columns)；drop刪除、passthrough保留
SequentialFeatureSelector	direction='forward' / 'backward'；n_features_to_select 指定欄數
SelectFromModel	依權重 / 重要度篩選；get_support() 的 True 為保留

概似最大化 ↔ log概似最大化 ↔ 負log概似最小化。log把連乘改連加、減少下溢。AI：鑑別式做分類 / 預測，生成式產生內容；Agent可規劃及使用工具。

4 梯度下降家族

	每次更新用	特性
Full-batch GD	全部樣本	方向穩、每步貴；每輪更新 **1 次**
SGD	1 筆	更新頻繁、路徑吵雜、可**線上學習**（`partial_fit`），每 epoch 要洗牌
Mini-batch	一小批 B	折衷，可向量化；每輪 $\lceil n/B \rceil$ 次（留末批）→ $n{=}100, B{=}20$ 是 **5 次**

- 更新：$\boldsymbol{w} \leftarrow \boldsymbol{w} - \eta\nabla L$（沿**負梯度**，梯度指向上升最快）
- 學習率太大 → 跨過谷底、loss 上升**發散**；太小 → 收斂很慢
- 自適應學習率 $\eta_t = \frac{\eta_0}{1+\alpha t}$：前期大步、後期小步
- **特徵縮放**讓等高線接近圓、GD 收斂快很多；標準化只平移縮放，分布形狀不變≠變成常態分布

5 資料前處理

- 前處理項目：清理、編碼、特徵衍生、縮放、降維（衍生、降維屬特徵工程）
- **黃金法則：先切分、再處理**；轉換器只對訓練集 `fit`，測試集只 `transform`
- **資料洩漏**：測試集資訊（平均、標準差…）滲進訓練 → 分數虛高
- 缺值：`isnull().sum()` 數每欄；刪 `dropna()` 或補（平均、最頻繁值＝類別型、KNN）；空字串 "" 不算缺值
- **順序型**（XL>L>M）→ 手動映射整數；**名義型**（顏色）→ 獨熱編碼，編 0/1/2 會造出假的大小關係
- 獨熱 k 類：含截距的線性模型可丟一欄剩 $k-1$ 欄，避免完全共線（全 k 欄和為 1，與截距相依）；不是所有模型都必須丟欄
- **特徵 X 與標籤 y 分清楚**：名義特徵可 one-hot；一般分類標籤只要一致對應，整數編號無大小意義。
- 課堂：one-hot 的 0/1 欄通常不必另縮放；不是所有應用都禁止縮放。
- 切分比例依資料量；分類用 `stratify=y`

	正規化 min-max	標準化 z-score
公式	$\frac{x-x_{\min}}{x_{\max}-x_{\min}} \to [0,1]$	$\frac{x-\mu}{\sigma}$ → 平均 0、標準差 1
離群值	**敏感**（其餘被擠扁）	仍受離群值影響；不限制範圍
類別	`MinMaxScaler`	`StandardScaler`（離群多用 `RobustScaler`）

標準化有助 GD / SGD，但仍受離群值影響。Min-max 訓練範圍 [0,1]，新資料可超出；抗極值可考慮 RobustScaler。

6 過擬合 / 欠擬合與正則化

	過擬合	欠擬合
別名	**高變異** high variance	**高偏差** high bias
症狀	訓練好、驗證差（有縫）	訓練、驗證都差
對策	加資料、加強正則化、減特徵、降複雜度	加特徵、更複雜模型、減弱正則化
	L_2（權重衰減）	L_1
懲罰	$\frac{\lambda}{2n}\sum w_j^2$	$\frac{\lambda}{n}\sum\|w_j\|$
效果	全部壓小、很少變 0	**稀疏**，很多權重恰為 0 → 可當特徵選取
幾何	約束區域是圓	菱形，角在座標軸上

- λ 越大越趨向欠擬合；sklearn 的 $C \propto 1/\lambda$：C 越小 = 正則化越強
- 正則化前通常先標準化（各特徵尺度一致，懲罰才公平）
- 較大模型訓練 loss 反而更高：先查最佳化（初始化、學習率），不能只判過擬合。
- **泛化誤差**＝在沒見過的資料上的誤差，用測試集估計；測試集越小估得越不準

補強 C｜評估函式與報告（L6）

precision_score / recall_score / f1_score	計算對應指標
matthews_corrcoef	MCC；1完美、0無相關、-1反向
classification_report	各類P / R / F1 / support及平均
roc_curve(y,p)	回fpr、tpr、thresholds
auc(fpr,tpr)	由曲線計面積
roc_auc_score(y,p)	由連續分數 / 正類機率計AUC
make_scorer(f1_score,pos_label=0)	自訂評分器，以0為正類

support＝真實該類樣本數；weighted依support加權。micro累加各類TP / FP / FN後算，完整單標籤多分類時micro P / R / F1＝accuracy。降低門檻→TPR、FPR不減，precision不保證下降。

7 特徵選取

- 序列特徵選取＝**貪婪搜尋**：每一步只挑當下最好的一欄，**前面丟掉的之後不再回頭考慮**，所以是次佳解；但避開了窮舉 2^d 組合的爆炸
- **SFS** 從空集合逐個**加入**；**SBS** 從全集逐個**剔除**使判準 J 損失最小者
- L_1＋`SelectFromModel`（L5 p56）：L_1 會把不重要的權重壓到**恰好 0**，等於順手做了選取

8 看圖判讀（觀念應用）

圖上長相	判讀＋為什麼
loss 對 epoch 越來越大、震盪	**學習率太大**：步伐跨過谷底、落到對面更高處，來回放大
loss 下降極慢、像直線	**學習率太小**每步移動太少；或**特徵沒縮放**讓等高線極扁，負梯度不指向谷底
SGD 的 loss 抖動但整體下降	**正常**：每步只用一筆估梯度，方向有雜訊但期望仍朝下坡
PLA 錯誤數最後歸零	**線性可分**：Novikoff 保證可分時有限步內收斂
PLA 錯誤數一直不歸零	**可能不可分**（永遠有分錯的樣本，更新停不下來）**或輪數不足**
訓練 acc 高、驗證 acc 低	**過擬合**：連訓練集的雜訊都記住了，新資料用不上
兩條 acc 都低且貼合	**欠擬合**：表達力不足，連訓練集的規律都抓不到
loss 收斂但驗證沒變好	**loss 收斂 ≠ 泛化變好**：loss 是訓練目標上的值，和 acc / F1 不是同一件事

9 易混淆一句話

① 邏輯迴歸損失 = 交叉熵，不是 MSE（MSE 是 Adaline） ② 感知器分對不更新 ③ 不可線性分 → PLA 不收斂 ④ 標準化 ≠ 常態化 ⑤ 對測試集 `fit` = 洩漏 ⑥ 訓練分數本身就低 ≠ 過擬合 ⑦ C 是正則化強度的**倒數** ⑧ L_1 稀疏、L_2 只壓小 ⑨ 本課 One-hot 的 0/1 欄**通常不另**縮放（L5 p24 注意事項） ⑩ 多標籤可用 OvR，softmax 給互斥多類 ⑪ $\boldsymbol{w}$ 定超平面方向（法向量）、b 定離原點遠近 → 只改 b 是平移邊界 ⑫ 激活函數的用途是引入非線性；Adaline 訓練用恆等函數算誤差，分類時仍用閾值 ⑬ 概似是**參數**的函數，不是參數的機率 ⑭ 變異＝換份資料結果差多少（一致性）、偏差＝系統性誤差 ⑮ 多重共線性 → 需要矩陣求逆的方法會病態 ⑯ 獨熱的代價：欄位膨脹、與截距完全共線

10 定義、資料型態與 Iris

- **Mitchell**：任務 T、效能 P、經驗 E —— P 隨 E 提升即「學習」
- **李宏毅**：學習＝找一個函式（參數化模型）。**一個 x 只能對應一個輸出**——這是函式的定義；多對一可以（不同 x 給同一答案），一對多不行，否則同一筆輸入會有兩個預測
- AI（目標）⊃ ML（手段：資料 → 知識）⊃ DL（多層神經網路）
- 學完的用途＝對新資料做**預測**（prediction / inference）

角度	分類	例
結構	結構化	表格、關聯式 DB
	半結構化	JSON、XML、HTML
	非結構化	文字、圖片、影音
數值型	連續	身高、房價
	離散	房間數
類別型	名義（無序）	血型 → one-hot
	順序（有序）	S/M/L → 映射整數

	監督	非監督	強化
回饋	標籤	無	獎勵（常延遲）
目標	預測	找結構	學動作策略

強化式的回饋是獎勵，不是逐筆正確標籤；獎勵可能延遲到整段行動結束。

Iris 資料集

- 150 筆、4 個連續特徵（花萼 / 花瓣長寬，cm）、3 類各 50 筆、無缺值
- 按類別排序（1–50 / 51–100 / 101–150）→ 訓練前要 **shuffle**：不洗的話前 100 筆只含兩類，切分或逐筆更新都會嚴重偏掉
- 非監督示範：遮掉標籤後分群；群數相同不代表分群一定正確

題庫判讀提醒

W1–11 / W6–8 課堂比較：min-max 易把其餘值擠扁，題庫選標準化較適合；但 z-score 也受離群值影響，真正要抗極值可用 RobustScaler。

工作流程（5 步迴圈）

選特徵＋收集樣本 → 選指標 → 選演算法訓練 → 驗證集評估 → 調超參數 / 換演算法（回到前面）。**測試集一開始就切開、全程不碰**；縮放 / 降維只用訓練集 fit。

考前重點與 API 補強版 第二面｜模型評估與 Python 程式判讀

「看程式猜輸出」：先寫出每個變數的 shape / 型別，再推結果

11 模型選擇與交叉驗證

- 狹義：**超參數調整**；廣義：**演算法選擇**（用巢狀 CV）
- **Holdout**：訓練 / 驗證 / 測試三份；缺點是結果對切法敏感
- **k 折**：輪流 1 折驗證、$k-1$ 折訓練，取 k 次平均（看平均＋標準差）；常用 $k=10$；資料少可取較大 k，多則較小以省計算
- **分層 k 折**：每折保持類別比例 → 分類優先；**LOOCV**：$k=n$（資料很少時）
- **巢狀 CV**：外層評估、內層挑超參數；例 5×2
- 選好超參數後，用**完整訓練集重訓**；測試集最後評估一次，不用來挑選

曲線	橫軸	回答
學習曲線	訓練資料量	多蒐集資料有沒有用
驗證曲線	某超參數值	該選多少（驗證分數峰值）

- 兩線都低且貼合 → 高偏差，**只加資料通常幫助有限**
- 訓練高、驗證低、中間有縫 → 高變異，**加資料可能改善**
- 若橫軸向右代表能力增加（如 C↑），train升、CV降 → 過擬合；λ↑時方向相反

搜尋	做法	取捨
Grid	窮舉組合	網格內保證找到；次數 $k\times\prod n_j$
Random	從分布抽固定次數	重要參數常只有少數；同預算探索更廣
Halving	少資源全評估、逐輪淘汰	節省資源；可能早淘汰好候選

12 混淆矩陣與指標

	預測陽	預測陰
真實陽 P	TP	FN（漏報）
真實陰 N	FP（誤報）	TN

- $\mathrm{ACC}=\frac{TP+TN}{全部}$，$\mathrm{ERR}=1-\mathrm{ACC}$
- $\mathrm{PRE}=\frac{TP}{TP+FP}$：「說是陽的有幾成真陽」
- $\mathrm{REC}=\mathrm{TPR}=\frac{TP}{TP+FN}$：「真陽抓到幾成」
- $\mathrm{FPR}=\frac{FP}{FP+TN}$；$\mathrm{F1}=\frac{2PRE\cdot REC}{PRE+REC}$（調和平均，兩者都高才高）
- **MCC** 用四格，$[-1,1]$，不平衡時比 F1 穩
- **ROC**：掃門檻畫 FPR(x)—TPR(y)；**AUC** 1 完美、0.5 瞎猜；量排序能力
- 題庫 W7–10 的常見取捨：門檻低 → recall↑、precision↓；嚴格說 recall 不減，precision 不保證下降；癌症篩檢重 **recall**，垃圾信過濾重 **precision**
- **微平均**每樣本等權（大類主導）；**宏平均**每類別等權（不平衡適用）
- 挑出某一格的樣本：`mask=(y_true==1)&(y_pred==0)` → `np.where(mask)[0]` 即 **FN 的索引**；陣列條件用 `&` / `|` 且**各加括號**
- **類別不平衡**：1% 陽性全猜陰也有 99% ACC → 改指標、過採樣、加權 `class_weight`

例：TP40 FN10 FP20 TN30 → ACC .70、PRE .667、REC .80、F1 .727、FPR .40

13 scikit-learn API

API	方法	例
轉換器	`fit` → `transform`（`fit_transform`）	`StandardScaler`、`SimpleImputer`、`OneHotEncoder`
分類器	`fit(X,y)` → `predict(X)`	`LogisticRegression`、`Perceptron`

```
sc = StandardScaler()
X_tr = sc.fit_transform(X_train) # ✔ 只 fit 訓練
X_te = sc.transform(X_test) # ✔ 沿用訓練的 μ、σ
sc.fit_transform(X_test) # ✘ 重估測試統計量
```

- 學到的參數結尾帶底線：`coef_` 權重、`intercept_` 偏差；建構子給的是超參數
- `transform` 的特徵數須與 `fit` 時相同
- **Pipeline**：訓練時各轉換器 `fit_transform`、最後估計器 `fit`；預測時只 `transform` 再 `predict` → 將原始資料與整條管線交給 CV，逐折 fit
- `make_pipeline(StandardScaler(), LogisticRegression())` 免命名；`Pipeline([('sc',…),('clf',…)])`
- Pipeline 調參用**步驟名＋雙底線＋參數名**：`clf__C`、`svc__gamma` / `svc__kernel`（L6 p37、p47 範例）；不是單底線。`make_pipeline` 自動用類別名小寫命名，所以同一個參數變成 `logisticregression__C`。
- `cross_val_score(pipe,X,y,cv=10)` 回傳 **10 個分數的陣列**；GridSearch 3×4 組 × cv=5 = 60 次 CV fit（預設重訓再 +1）；`gs.best_estimator_` 取最佳模型，最後只在測試集 score 一次
- `train_test_split(…, test_size=0.3, stratify=y, random_state=1)`
- `joblib.dump` / `load` 存取**整條管線**（含前處理學到的統計量），載入後直接 `predict`，不必再訓練

14 Python 基礎（猜輸出）

```
7/2 → 3.5 # / 永遠回 float
7//2 → 3   -7//2 → -4 # 往負無窮取整，不是砍小數
7%3 → 1    -7%2 → 1 # 餘數符號跟除數走
2**3 → 8 # ** 是次方（^ 才是 XOR）
"3"*3 → '333' # str×int＝把字串重複 3 次再接起來
[1,2]*2 → [1,2,1,2] # list 整串重複，不是逐元素乘
"3"+3 → TypeError # 不自動轉型
a=[0,1,2,3,4,5] # a[start:stop:step] 含頭不含尾
a[1:4] → [1,2,3]  a[-2:] → [4,5] # -1 是最後一個
a[::2] → [0,2,4]  a[::-1] → [5,4,3,2,1,0] # 反轉
list(range(1,10,3)) → [1,4,7] # 1→4→7，10 已達 stop
[x**2 for x in range(5) if x%2==0] → [0,4,16]
# 先迭代→再 if 篩→最後算式子
len(set("hello")) → 4 # 逐字元去重 {h,e,l,o}
bool([]) → False # 空容器、0、""、None 皆為假
d={'a':1}; d.get('b',0) → 0; d['b'] → KeyError
# get 回預設值不拋錯，中括號才拋
t=(1,2); t[0]=5 → TypeError
# tuple 不可變：不能改元素，但可整個重新指派 t=(5,2)
f"{3.14159:.2f}" → '3.14' # 四捨五入，回「字串」
int("3.5") → ValueError # 要 int(float("3.5"))=3
def f(x, L=[]): L.append(x); return L
f(1); print(f(2)) → [1, 2]
# 預設值在 def 時只建一次，之後每次共用同一個
```

容器	可變	有序	用途
list	✔	✔	增刪暫存；依位置取 O(1)、找值 O(n)
tuple	✘	✔	固定組合；不可變才可雜湊，故能當 dict 的鍵
dict	✔	插入順序	依名字查；雜湊表，平均 O(1)
set	✔	✘	去重、交集 `&`；同樣靠雜湊，故無序

- `if/elif` 只執行第一個成立分支（FizzBuzz 要先判 15）；縮排是語法
- `__init__` 建構子；`self` = 物件本身；`self.x=x` 才存成屬性
- 選容器看可變性、順序、查詢成本；dict / set 用雜湊，平均查詢快。
- `input()` 回傳字串；`random.randint(1,100)` 兩端都含
- `try/except ValueError` 捕捉轉型錯誤；`PermissionError` 是 `OSError` 子類
- **os 模組**（L2 第 5 題）：`os.path.exists/isdir` 判斷路徑、`os.makedirs` 建資料夾、`os.listdir` 列出檔名，外面包 `try…except`
- 內建函式 `max()` / `min()` / `sum()`（L2 第 4 題配 `random` 模組與列表推導式）

15 NumPy

```
# .shape 是「每一維長度」組成的 tuple；由最外層往內數
np.array([1,2,3]).shape → (3,)
# 一層括號3個元素。一維就是一維，非行也非列向量
np.array([[1],[2],[3]]).shape → (3,1)
# 外層3個、每個內層1個 → 3列1欄的直欄
np.array([[[1],[2]]]).shape → (1,2,1)
# ndim=中括號層數；最外1、中間2、最內1
np.zeros((2,3)) : 全0；np.ones((2,3)) : 全1
# 傳進去的 tuple 直接就是 shape
np.eye(3) : 單位矩陣 # 對角線1、其餘0
(10,3,4)@(4,5) → (10,3,5)
# 只有末兩維做矩陣乘 (3,4)@(4,5)=(3,5)，10 當批次帶著走
A=np.array([[1,2],[3,4]]) # 兩個 2×2 方陣
B=np.array([[5,6],[7,8]])
A*B → [[5,12],[21,32]] # 逐元素：1×5、2×6、3×7、4×8
A@B → [[19,22],[43,50]] # =np.matmul；列點乘欄 1×5+2×7=19
np.dot([1,2,3],[4,5,6]) → 32 # 一維即內積 1×4+2×5+3×6
M=np.arange(6).reshape(2,3)
# arange(6)=[0..5] 共6個，再排成2列3欄 [[0,1,2],[3,4,5]]
M.sum(axis=0) → [3,5,7] # axis=被壓掉那維；同欄 0+3,1+4,2+5
M.sum(axis=1) → [3,12] # 同列 0+1+2, 3+4+5
x=np.array([1,5,3,8])
x>2 → [F,T,T,T] # 比較是逐元素的，得同長度布林陣列
x[x>2] → [5,3,8] # 布林遮罩：只留 True 的位置
x[[0,3]] → [1,8] # 花式索引：位置寫成清單，順序由清單決定
np.array([1,2.5]).dtype → float64 # 同型別，int 升 float
x[:,np.newaxis].shape → (4,1) # 變直欄，同 reshape(-1,1)
```

廣播：從**最右邊**維度往左比，每對須**相等或其一為 1**；缺的維度補 1；結果取各維最大。

形狀	結果
(4,3)+(3,)	(4,3)　右邊左補成 (1,3)；3=3、1 撐成 4
(4,3)+(4,)	ValueError
補成 (1,4) 後最右 3 vs 4，兩者都不是 1 → 失敗。要成功先 `reshape(-1,1)`	
(4,1)+(3,)	(4,3)　(4,1) 與 (1,3)，兩邊各把 1 撐開
(3,1,2)−(5,2)	(3,5,2)　2=2、1 撐成 5、左補 1 給 3（座標差，還不是距離）
(8,1,6,1)+(7,1,5)	(8,7,6,5)　右邊補成 (1,7,1,5)，逐位取大
(n,)+(n,1)	(n,n)：不報錯卻算錯！
(n,) 補成 (1,n) 被當成一「列」、另一個是直欄，撐成方陣	

- `axis=k` 表示**被壓掉**的那一維——**axis 寫幾，shape 的第幾個數字就消失**（標準化的 `X.mean(axis=0)` 就是「每個特徵欄各一個平均」）；`.T` 對一維陣列無效（只有一個軸，沒有兩軸可換），要直欄得用 `reshape(-1,1)` 或 `np.newaxis`
- **距離矩陣**：A:(a,m)、B:(b,m) → `A[:,None,:]-B[None,:,:]`，即 (a,1,m) 與 (1,b,m) 廣播成 (a,b,m)＝每對點的**座標差**；再沿 `axis=2` 取 norm 壓掉 m 才是 (a,b) 距離
- **同步洗牌**：`idx=rng.permutation(len(y))`、`X,y=X[idx],y[idx]` 必須用**同一組索引**
- `b=a` 只是多一個名字指向**同一塊資料**，改 b 等於改 a；`b=a.copy()` 才另配一份。`np.where(cond,1,0)` 是三參數版：條件成立取 1、否則取 0，逐元素套用
- ndarray 元素同型別；切片可能不連續；向量化由預編譯底層常式處理，減少 Python 迴圈；矩陣乘法用 `@`
- **線上學習**：課堂 `AdalineSGD.partial_fit` 保留原權重，用新資料接續更新；`fit` 重新初始化訓練。

16 Pandas

`pd.read_csv` 讀 CSV（L5 p9、p71）；`pd.Series(data,index=...)` 建一維帶標籤資料；多個 Series 組 DataFrame 依索引對齊。

- DataFrame＝共用索引的多個 Series；`df['col']` → Series；`df[['col']]` → 二維 DataFrame
- **轉成 NumPy**：`Series.to_numpy()` / `DataFrame.to_numpy()`（L3 p48、p50 的重點）
- `isnull()`（NaN/None/NaT）、`dropna()` 預設**刪列**、`axis=1` 刪欄、`fillna(v)`
- `pd.get_dummies(…, drop_first=True)` → $k-1$ 欄
- **課堂的取欄寫法**（L5 p71 端到端範例）：`X = df.drop(columns=['target']).to_numpy()`、`y_raw = df['target'].to_numpy()`；`LabelEncoder` 轉完用 `classes_` 看對照（B→0、M→1）

補強 B｜搜尋、回傳值與評估（L6）

StratifiedKFold.split	回各折訓練 / 驗證索引；shuffle 預設False
learning_curve	回 train_sizes、train_scores、test_scores
validation_curve	回 train_scores、test_scores；改 param_range
GridSearchCV	param_grid窮舉；多字典分別算組合後相加
RandomizedSearchCV	param_distributions；n_iter抽樣組數
HalvingRandomSearchCV	逐次加資源、淘汰候選；factor 決定淘汰比例

曲線函式 test_scores 指 CV 驗證折，不是最終 test。n_jobs=−1 用全部 CPU；scoring 選評分標準，不改訓練 loss。best_params_最佳設定、best_score_平均CV最佳分數；refit=True時 best_estimator_ 已重訓。

17 Matplotlib

- 狀態式 `plt.plot/plt.title`（作用在「目前的圖」）vs 物件導向 `fig, ax = plt.subplots()`、`ax.set_title`（多出 `set_`）
- **Figure**＝整張畫布；**Axes**＝子圖；Axis＝單條軸。
- `axes[0,2]` 從 0 起算 = `plt.subplot(2,3,3)` 從 1 起算、逐列編號
- `plot` 全點同樣式、快；`scatter` 可逐點設 `s` / `c` / `alpha`、慢

圖	函式	用途
折線	`plt.plot()`	追蹤隨時間 / 序列變化
散點	`plt.scatter()`	看兩變數的相關
長條	`plt.bar()`	比較不同類別的值
直方	`plt.hist()`	看資料分布
圓餅	`plt.pie()`	看占整體的比例

Common Functions（L4 p33）：`plt.style.use('seaborn-v0_8-poster')` 設樣式、`plt.tight_layout()` 調子圖間距、`plt.show()` 顯示、`plt.savefig("file_name.png")` 存檔。

18 模型內部功能辨識

各片段獨立辨識，對照課堂 0/1 分類；`np` 為 NumPy。

```
z = X @ w + b
# 淨輸入；X:(n,d), w:(d,) → z:(n,)
y_hat = (z >= 0).astype(int)
# PLA 的階梯分類：z=0 也輸出 1
pred = int(xi @ w + b >= 0)
update = eta * (target - pred)
w += update * xi; b += update
# PLA 單筆更新：分對 → update=0
a = z
# Adaline 訓練用線性激活
a = 1 / (1 + np.exp(-z))
# 邏輯迴歸 sigmoid；不是類別標籤
errors = y - a
w += eta * X.T @ errors / len(y)
# 全批次更新；a 的定義決定模型
b += eta * errors.mean()
```

程式 / 線索	辨識重點
`((y-a)**2).mean()`	MSE；Adaline 的 a 是線性輸出
出現 `y*np.log(a)`	BCE；a 是機率
外層 epoch、內層逐筆更新	若依連續誤差 / 梯度更新為 SGD；PLA 也有逐筆迴圈
每輪對整個 X 算梯度一次	Full-batch；每輪更新 1 次

```
X=np.array([[1.,2.],[3.,4.]])
w=np.array([1.,-1.]); b=0.
z=X@w+b → [-1.,-1.] # X@w 算完要加 b 才是 z
(z>=0).astype(int) → [0,0] # True→1、False→0
# 同一筆換 b=1. : z=[0,0]
p=1/(1+np.exp(-z)) → [.5,.5]
pred=(p>=.5).astype(int) → [1,1]
eta=0.1; target=1; pred=0
u=eta*(target-pred) → 0.1
w+u*X[0] → [1.1,-0.8]
b+u → 0.1
```

題目要機率就回 `p`、要類別才回 `pred`，別混用。MSE 代入例：$y=[1,0]$、$a=[.8,.3]$ → $e=[.2,-.3]$ → $\frac{.04+.09}{2}=.065$。

更新係數別硬背，先看損失定義。Adaline 損失是 $\frac{1}{n}\sum e^2$ 時 `w += eta*2*(X.T@e)/len(y)`、`b += eta*2*e.mean()`；題目若寫 $\frac{1}{2n}\sum e^2$ 就**少一個 2**。邏輯迴歸 BCE 改成 `e = y - p` 且**沒有那個 2**。

BCE 實作：防 `log(0)` 的兩種等價寫法

```
p = np.clip(p, 1e-15, 1-1e-15)
loss = -np.mean(y*np.log(p) + (1-y)*np.log(1-p))
# @ 對一維陣列＝逐項相乘後加總，除樣本數即平均
loss = (-y @ np.log(p) - (1-y) @ np.log(1-p)) / len(y)
```

`y` 與 `p` 都應為 `(n,)`，別把閾值化後的 `pred` 當 `p` 代入。單筆 $y=1$：$p=0.9$ → 0.105、$p=0.1$ → 2.303，**分對類別不代表 loss=0**。迴圈：X、y 各 4 筆、`range(3)` 內層逐筆 → 共 **12 筆次**，但 PLA 實際更新可少於 12 次；下一筆沿用前一筆更新後的參數。

19 API 結果預期與常見陷阱

predict 回傳類別 (n,)；**predict_proba** 回傳 (n,K)，欄位順序看 `classes_`，不是預測正確率。

```
# 假設 clf.classes_ 為 [0,1]
P=np.array([[.8,.2],[.3,.7],[.6,.4]])
P[:,1] → [.2,.7,.4]（類別1機率）
(P[:,1]>=.5).astype(int) → [0,1,0]
(P[:,1]>=.3).astype(int) → [0,1,1]
# StandardScaler：以訓練統計量轉換
sc=StandardScaler().fit([[0.],[2.]])
sc.mean_ → [1.]
sc.transform([[3.]]) → [[2.]]
```

- 100 筆、`test_size=.3` → 訓練 70、測試 30；回傳順序 `X_train,X_test,y_train,y_test`。
- `stratify=y` 保持類別比例；`random_state` 固定切分的隨機性，不保證分數更高。
- `confusion_matrix` 的列＝真實、欄＝預測；labels=[0,1] 時為 `[[TN,FP],[FN,TP]]`，與 §12 排列不同。
- 更新 `w` 之後，**先前算好的 `output` / `e` 不會自動重算**；`errors` 這類計數器須事先初始化。
- 判讀順序：**型別 → shape → 索引 → 運算 → 回傳值**。